

MACHINE LEARNING OPERATIONS

Credit Default Prediction with MLOps

Fabício Olo

Hugo Lima

Maria Teresa Neves

Olívia Rua

Pedro Martim Lota

Executive Master in Business Analytics & AI

Challenge

Which customers will miss their next payment?



Banks often find out about a default when it is too late to help.



We predict the probability that each customer defaults next month.



An analyst reviews the flagged customers. The model does not decide.

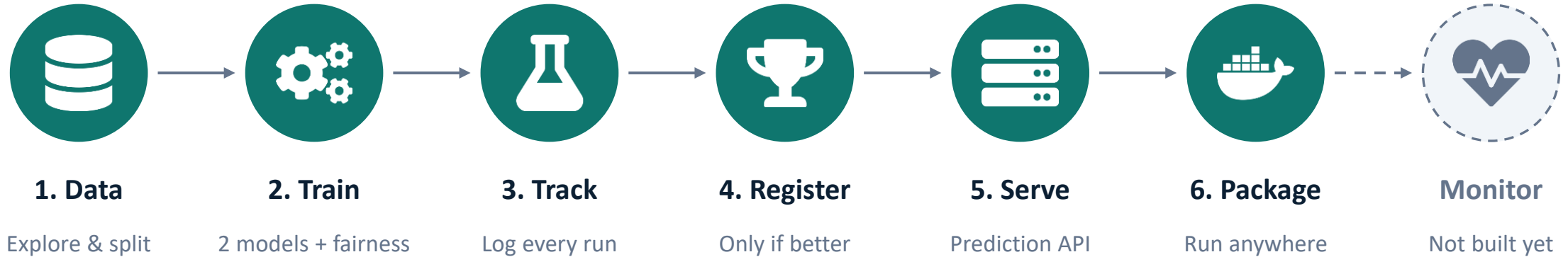
DATASET

Default of Credit Card Clients (Kaggle / UCI)

- 30,000 customers, Taiwan 2005
- 23 features: credit limit, age, 6 months of payments and bills
- Target: default next month (yes / no)

HOW WE BUILT IT

Our pipeline and the tools we used



TOOLS

pandas

scikit-learn

MLflow
Tracking

MLflow
Model Registry

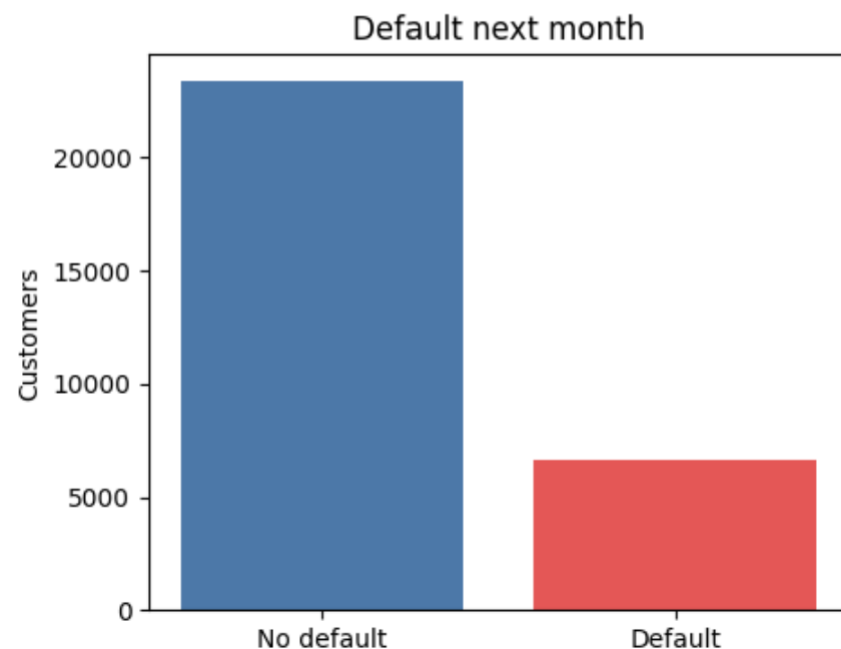
FastAPI

Docker Compose
pytest & GitHub CI

e.g. Evidently

Exploring the data

	customers	share
default.payment.next.month		
No default	23364	0.779
Default	6636	0.221



- Only 22% of customers default
- Understanding this is crucial to interpret metrics and design the model, as classes are imbalanced.
- We used a stratified split: train and test keep the same 22%



1. Data



2. Train



3. Track



4. Register



5. Serve



6. Package



Monitor (next)

Training two models with one command

```
> docker compose --profile train run --rm trainer
```

```
===== MODEL COMPARISON =====  
metric          logistic_regression      random_forest  
roc_auc          0.7446                          0.7751  
recall           0.5614                          0.5803  
precision        0.4703                          0.5036  
f1               0.5119                          0.5392  
accuracy         0.7632                          0.7807  
=====
```

- Two models: Logistic Regression and Random Forest
- Cleaning, features and model are saved together in one pipeline, so the API prepares new data the same way
- Every run also checks fairness per group
- Fixed random seed: same results every time



1. Data



2. Train



3. Track



4. Register



5. Serve



6. Package



Monitor (next)

Tracking the experiments in MLflow

Run Name	Created	Duration	recall	roc_auc
random_forest	1 hour ago	7.4s	0.580256217...	0.775110218...
logistic_regression	1 hour ago	6.8s	0.561416729...	0.744590087...

Compare view: both runs side by side

model_type	random_forest	logistic_regression
n_estimators	200	
n_raw_features	23	23
random_state	42	42
accuracy	0.781	0.763
f1	0.539	0.512
precision	0.504	0.47
recall	0.58	0.561
roc_auc	0.775	0.745

- Every training run is saved automatically
- Each run keeps its parameters, metrics, charts and the model file
- We can compare runs side by side instead of keeping notes



1. Data



2. Train



3. Track



4. Register



5. Serve

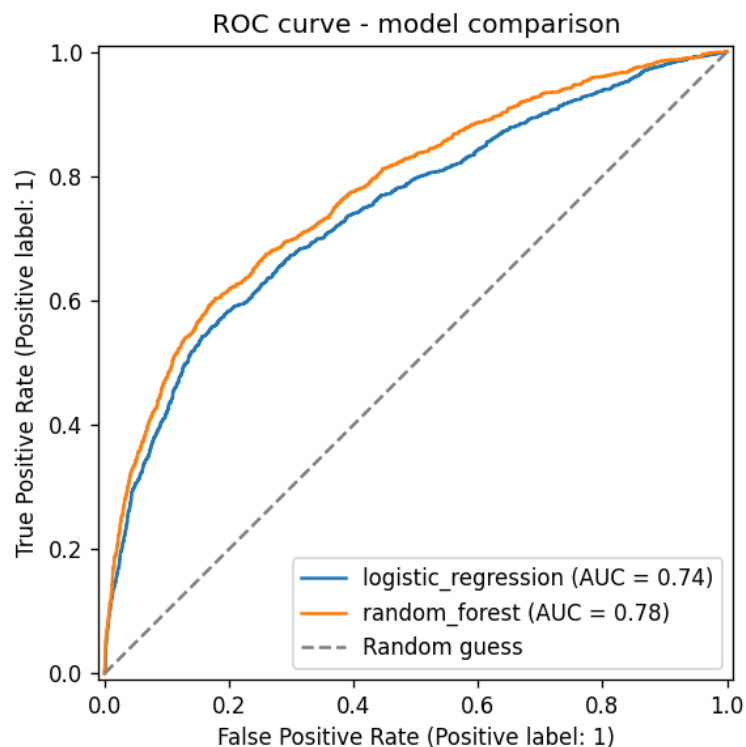


6. Package



Monitor (next)

Champion vs Challenger: only a better model goes live



ROC-AUC

Random Forest 0.775

Logistic Regr. 0.745

The best model is the challenger. It becomes @champion only if ROC-AUC beats the current champion.

✓ Version 3	09/29/2026, 11:34:30 PM	model_type: random_forest source_run_id: 7ae9b046c6344ce58d promotio... : not promoted: roc_auc	@ challenger	random_forest, ROC-AUC = 0.77...
✓ Version 2	09/29/2026, 11:24:08 PM	model_type: random_forest source_run_id: 78350b7dfc1c45a497	@ champion	random_forest selected by ROC...

@champion

v2 · the model the API serves

@challenger

v3 · same score, so not promoted



1. Data



2. Train



3. Track



4. Register



5. Serve

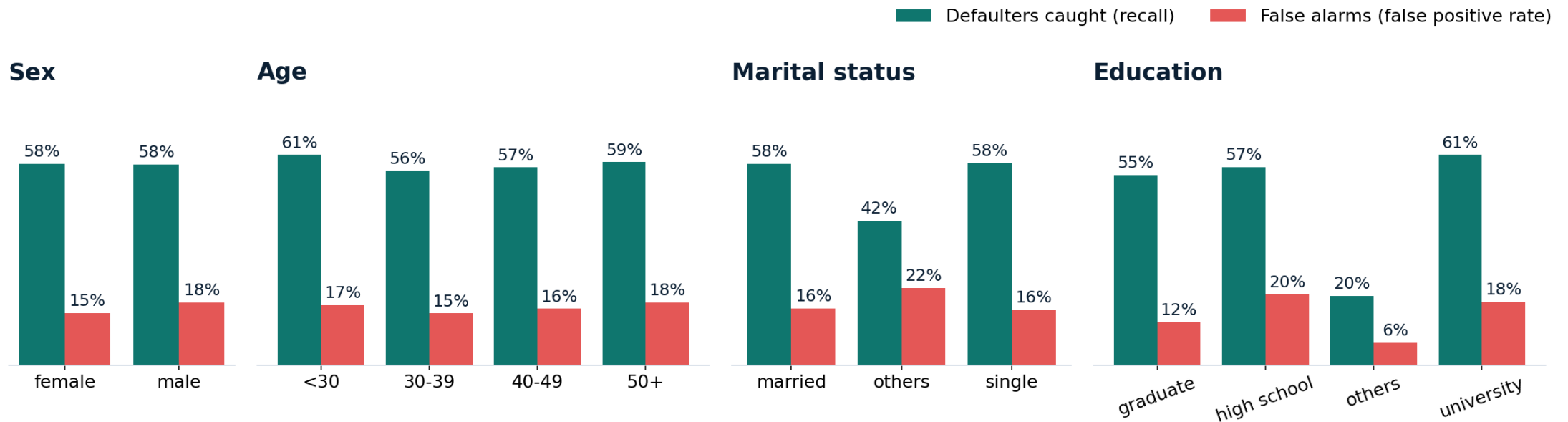


6. Package



Monitor (next)

Is the model fair? Checked on every training run



Sex

women and men: 58% of defaulters caught

Age

all age bands within the 80% rule

Marital status

small 'others' group gets more false alarms

Education

false alarms: high school 20%, graduate 12%



1. Data



2. Train



3. Track



4. Register



5. Serve



6. Package



Monitor (next)

Serving predictions through an API

Valid customer → prediction (real response from our API, Swagger)

```
{
  "prediction": 1,
  "label": "high_default_risk",
  "default_probability": 0.7704,
  "threshold": 0.5,
  "model_version": "1",
  "model_uri": "models:/credit_default_model@champion",
  "disclaimer": "Academic decision-support prototype. The score must be reviewed by a human and must not be used as an automated credit decision."
}
```

high_default_risk

label

0.77

default probability

version 1

model that answered

- The API loads the @champion model
- Wrong input is rejected before it reaches the model
- Every answer says a human must review the score

Wrong input (AGE = 150) gets rejected

```
422 Error: Unprocessable Entity
Response body
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": [
        "body",
        "AGE"
      ],
      "msg": "Input should be less than or equal to 100",
      "input": 150,
      "ctx": {
        "le": 100
      }
    }
  ]
}
```



1. Data



2. Train



3. Track



4. Register



5. Serve



6. Package



Monitor (next)

Running everything with Docker

Docker Desktop: our containers running, with their live logs

mlflow ● credit-default-ml○ 5000:5000 ↗	mlflow	<pre> 2026/09/26 19:29:17 INFO: 172.18.0.1:39752 - "POST /ajax-api/2.0/mlflow/runs/search HTTP/1.1" 200 OK 2026/09/26 19:29:20 INFO: 127.0.0.1:56338 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:26 INFO: 127.0.0.1:56344 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:31 INFO: 127.0.0.1:35740 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:36 INFO: 127.0.0.1:35746 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:41 INFO: 127.0.0.1:35228 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:46 INFO: 127.0.0.1:35244 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:52 INFO: 127.0.0.1:33650 - "GET /health HTTP/1.1" 200 OK 2026/09/26 19:29:57 INFO: 127.0.0.1:33662 - "GET /health HTTP/1.1" 200 OK </pre>
api ● credit-default-ml○ 8000:8000 ↗	api	<pre> INFO: 172.18.0.1:51818 - "GET /docs HTTP/1.1" 200 OK INFO: 172.18.0.1:51818 - "GET /openapi.json HTTP/1.1" 200 OK </pre>

mlflow

tracking + registry · port 5000

api

prediction service · port 8000

trainer

runs once when we train, then removed

A teammate needs only Git, Docker and the CSV. 27 automated tests + a Docker build run on every push to GitHub (CI green).



1. Data



2. Train



3. Track



4. Register



5. Serve



6. Package



Monitor (next)

What we built and what comes next

What we built

- Experiments + fairness tracked in MLflow
- Champion vs Challenger registry
- Prediction API with input checks
- Docker + 27 tests in GitHub CI

Next steps

- Monitor drift over time (Evidently)
- Retrain without sensitive attributes
- Set the threshold with the business
- Cross-validation and tuning

Responsible use

- Sex and age pass; education under review
- A human reviews every flag
- Model card documents the limits
- High-risk under the EU AI Act; GDPR